

Universidade Estadual do Ceará
Centro de Ciências e Tecnologia
Curso de Bacharelado em Ciência da Computação

Extensão III

EcoRoteiro - Plataforma de Ecoturismo Inteligente

Jamile Hellen - 1625580
Marianna Onofre - 1625566
Vilma Araújo - 1625618
Yanne Oliveira - 1623800

Fortaleza - Ceará
2025

SUMÁRIO

1. Descrição do Sistema
2. Objetivo do Documento
3. Escopo do projeto
4. Público Alvo
5. Glossário Técnico
6. Requisitos Funcionais
7. Requisitos Não Funcionais
8. Regras de Negócio
9. Arquitetura Técnica e Fluxo de dados
10. Modelagem de dados
11. Especificação da API

1. Descrição do Sistema

O **EcoRoteiro** é uma plataforma web de ecoturismo inteligente que utiliza Inteligência Artificial Generativa (GenAI) para criar roteiros personalizados de turismo ecológico no estado do Ceará. O sistema visa democratizar o acesso ao turismo sustentável, oferecendo sugestões curadas e adaptadas às características individuais do usuário, como interesses, condição física e nível de experiência.

Diferente de chatbots genéricos, o EcoRoteiro utiliza uma arquitetura **RAG (Retrieval-Augmented Generation)**, combinando a fluidez de um Modelo de Linguagem (LLM) com a precisão de uma base de dados vetorial curada, garantindo recomendações seguras e fundamentadas.

2. Objetivo do Documento

Este documento tem como objetivo descrever formalmente os requisitos funcionais, não funcionais e regras de negócio do sistema EcoRoteiro. Ele serve como guia para o desenvolvimento do protótipo acadêmico (MVP), documentando o escopo, as restrições e as funcionalidades esperadas.

3. Escopo do Projeto

3.1. O que o sistema FAZ (In-Scope):

- **Gestão de Identidade:** Cadastro e autenticação segura de usuários.
- **Perfilamento:** Coleta de dados sobre condição física, nível de experiência (Iniciante, Intermediário, Avançado) e interesses (ex: Trilhas, Praias, Parques).
- **Chatbot Especialista:** Interface conversacional focada exclusivamente em Ecoturismo no Ceará.
- **Geração de Roteiros (RAG):** Busca semântica de locais em base própria e geração de descrições via IA.
- **Defesa Cognitiva:** Mecanismos de bloqueio para solicitações fora do escopo geográfico (outros estados) ou temático (assuntos não turísticos).

3.2. O que o sistema NÃO FAZ (Out-of-Scope):

- Reservas ou compra de passagens.
- Integração com mapas em tempo real (GPS/Navegação turn-by-turn).
- Funcionalidades de rede social (compartilhamento entre usuários).
- Cobertura de turismo urbano não-ecológico (shoppings, cinemas).

4. Público-Alvo

- **Turistas e Residentes:** Interessados em redescobrir as belezas naturais do Ceará.
- **Praticantes de Atividades ao Ar Livre:** Pessoas que buscam trilhas e parques seguros.
- **Iniciantes no Ecoturismo:** Pessoas que desejam começar a explorar a natureza, mas precisam de curadoria de segurança adequada ao seu nível físico.

5. Glossário Técnico

- **RAG (Retrieval-Augmented Generation):** Técnica que une a busca em banco de dados com a geração de texto por IA para aumentar a precisão.
- **LLM (Large Language Model):** Modelo de linguagem de grande escala (no caso, Llama 3) usado para interpretar a linguagem natural.
- **Prompt Injection:** Tentativa de manipular a IA para realizar tarefas não autorizadas.
- **Vector Database (ChromaDB):** Banco de dados otimizado para buscar similaridade semântica entre textos.
- **Perfil de Usuário:** Conjunto de metadados (interesses, nível) usados para filtrar as recomendações.

6. Requisitos Funcionais (RF)

Funcionalidades que o sistema deve oferecer ao usuário.

ID	Requisito	Descrição	Prioridade
RF01	Manter Perfil do Usuário	O sistema deve permitir que o usuário defina e edite seu nível de experiência, condição física e interesses.	Alta
RF02	Chatbot Contextual	O sistema deve oferecer uma interface de chat onde o usuário possa pedir sugestões em linguagem natural.	Alta
RF03	Recomendação Baseada em RAG	Ao receber uma pergunta, o sistema deve buscar roteiros compatíveis na base de dados vetorial antes de responder.	Alta
RF04	Bloqueio de Conteúdo (Guardrails)	O sistema deve interceptar e recusar solicitações sobre locais fora do Ceará ou temas não relacionados (ex: culinária, código).	Alta
RF05	Sugestão Estruturada	O sistema deve ser capaz de gerar cards de sugestão contendo Título, Descrição, Dificuldade e Destaques.	Média
RF06	Autenticação Segura	O sistema deve permitir cadastro e login via e-mail e senha.	Alta

7. Requisitos Não Funcionais (RNF)

Critérios de qualidade, desempenho e restrições técnicas.

ID	Requisito	Descrição
RNF01	Latência de Inferência	A geração de respostas pela IA deve ocorrer, em média, em menos de 3 segundos (garantido pela arquitetura Groq LPU).
RNF02	Segurança de Navegação	O Frontend deve implementar <i>Route Guards</i> para impedir acesso não autorizado a páginas internas.
RNF03	Confiabilidade (Grounding)	A IA deve priorizar informações da base local (ChromaDB) para minimizar alucinações factuais.
RNF04	Arquitetura	O sistema deve ser desacoplado (Frontend React e Backend Python FastAPI).
RNF05	Acessibilidade	A interface deve seguir princípios básicos de contraste e legibilidade.

8. Regras de Negócio (RN)

Lógicas de decisão implementadas no Backend.

- **RN01 – Restrição Geográfica Rigorosa:** O sistema atua exclusivamente no território do **Ceará**. Solicitações para outros estados (ex: Gramado, Rio de Janeiro) devem ser identificadas e redirecionadas para sugestões locais equivalentes (Pivotagem).
- **RN02 – Filtragem de Segurança por Nível:** Usuários com perfil "Iniciante" **não** devem receber recomendações de trilhas classificadas como "Difícil", independentemente da solicitação.
- **RN03 – Escopo Temático:** O assistente deve recusar solicitações que fujam do tema "Ecoturismo e Natureza" (ex: pedidos de receitas, resolução de cálculos matemáticos ou programação).

- **RN04 – Obrigatoriedade do Perfil:** Para receber recomendações personalizadas, o usuário deve ter completado o cadastro de interesses obrigatórios.
- **RN05 – Privacidade:** As conversas e dados do perfil são privados e acessíveis apenas pelo próprio usuário autenticado.

9. Arquitetura Técnica e Fluxo de Dados

Esta seção detalha a composição modular do sistema, justificando as escolhas arquiteturais e descrevendo o fluxo de dados desde a interação do usuário até a inferência da Inteligência Artificial.

9.1. Diagrama de Arquitetura Geral

O sistema segue uma arquitetura cliente-servidor desacoplada, onde o Frontend (React) consome serviços de uma API RESTful (FastAPI), que por sua vez orquestra serviços de terceiros (Supabase, Groq).

9.2. Stack Tecnológica Detalhada

Abaixo detalha-se a composição modular do backend, justificando a escolha de cada biblioteca em função de sua responsabilidade no sistema.

A. Motor de Inferência (LLM Interface)

- **Biblioteca:** groq
- **Modelo:** llama-3.3-70b-versatile
- **Função Técnica:** Atua como o núcleo de processamento de linguagem natural. O sistema submete o input do usuário associado ao contexto recuperado (RAG) para a API da Groq.
- **Justificativa:** A escolha da Groq deve-se à sua arquitetura LPU (*Language Processing Unit*), que oferece baixa latência na inferência, garantindo fluidez na interação em tempo real.

B. Gateway de API e Servidor de Aplicação

- **Bibliotecas:** fastapi, uvicorn
- **Função Técnica:**
 - **FastAPI:** Gerencia o roteamento de requisições HTTP, expondo endpoints RESTful (ex: /chat, /auth) para comunicação com o frontend.

- **Uvicorn:** Atua como servidor ASGI (*Asynchronous Server Gateway Interface*), responsável pela execução da aplicação em ambiente de produção/desenvolvimento.

C. Camada de Persistência Relacional

- **Biblioteca:** supabase (cliente Python)
- **Função Técnica:** Gerenciamento de dados estruturados e persistentes. Responsável pela autenticação e armazenamento de perfis de usuário (dados demográficos, nível de experiência e interesses), garantindo a integridade referencial.

D. Recuperação Semântica e Banco Vetorial (RAG)

- **Bibliotecas:** chromadb, sentence-transformers
- **Função Técnica:** Implementação da arquitetura RAG (*Retrieval-Augmented Generation*).
 - **Sentence-transformers:** Realiza a tokenização e geração de *embeddings* (vetores numéricos) a partir das consultas em linguagem natural.
 - **ChromaDB:** Armazena e indexa os vetores dos roteiros turísticos, permitindo busca por similaridade semântica (cosseno) em vez de correspondência exata de palavras-chave.

E. Validação e Serialização de Dados

- **Biblioteca:** pydantic
- **Função Técnica:** Assegura a integridade dos dados trafegados (*Data Validation*). Define esquemas rígidos (Schemas) para payloads de entrada e saída, prevenindo erros de tipagem em tempo de execução.

9.3. Fluxo de Processamento da Requisição (Pipeline)

O ciclo de vida de uma interação do usuário segue um fluxo lógico sequencial, garantindo segurança e contexto antes da resposta.

1. **Intercepção:** O FastAPI recebe a requisição do cliente via HTTPS.
2. **Autenticação:** O sistema valida o token de sessão do usuário via Supabase.
3. **Filtro de Segurança (Python Guard):** O código verifica se há termos proibidos (ex: outros estados) antes de prosseguir.
4. **Recuperação Vetorial:** O ChromaDB consulta a base de conhecimento (JSON indexado) em busca de trechos semanticamente relevantes à consulta do usuário.

5. **Agregação de Contexto:** O sistema compila um *prompt* único contendo: a consulta original + o perfil do usuário (Supabase) + os dados recuperados (ChromaDB).
6. **Inferência:** O *prompt* enriquecido é enviado à Groq, que gera a resposta contextualizada.
7. **Resposta:** O FastAPI retorna a saída estruturada para o frontend.

9.4. Engenharia de Prompt e Mecanismos de Segurança

O sistema utiliza técnicas avançadas de *Prompt Engineering* para garantir alinhamento, segurança e personalização. A instrução do sistema é estruturada em camadas lógicas de controle, conforme ilustrado abaixo.

- **3.1. Camada de Identidade (Role Prompting):**
 - *Técnica:* Definição de Persona e Restrição Cognitiva.
 - *Aplicação:* O sistema é instruído a assumir a identidade de um "Especialista em Ceará", com a restrição explícita de desconhecer outras geografias. Isso mitiga alucinações sobre domínios fora do escopo do projeto.
- **3.2. Restrições Negativas (Negative Constraints):**
 - *Técnica:* Bloqueio Geográfico e de Domínio.
 - *Aplicação:* Instruções imperativas que definem o que a IA não deve fazer. Exemplo: "Se o input contiver solicitações sobre outros estados, decline a resposta." Isso garante a fidelidade ao propósito do produto.
- **3.3. Guardrails de Tópico (Topic Boundaries):**
 - *Técnica:* Proteção contra Desvio de Escopo (*Prompt Injection Defense*).
 - *Aplicação:* O sistema recusa interações que não sejam sobre Ecoturismo (ex: pedidos de código, matemática ou culinária).
- **3.4. Fundamentação em Dados (Grounding/RAG):**
 - *Técnica:* Context Injection.
 - *Aplicação:* O *prompt* prioriza explicitamente o contexto injetado (CONTEXTO TÉCNICO) sobre o conhecimento pré-treinado do modelo. A diretiva é: "Use os dados fornecidos; se ausentes, use conhecimento geral restrito ao Ceará". Isso reduz a taxa de alucinação factual.
- **3.5. Personalização Dinâmica:**
 - *Técnica:* Injeção de Variáveis de Usuário.
 - *Aplicação:* O *prompt* é adaptado em tempo de execução com variáveis (`{nível}`, `{interesses}`), permitindo que o modelo ajuste o tom e a complexidade da resposta baseando-se no perfil do usuário armazenado no banco relacional.

10. Modelagem de Dados

Descrição das estruturas de armazenamento utilizadas no sistema, divididas entre Relacional (Supabase) e Vetorial (ChromaDB).

10.1. Banco Relacional (Supabase/PostgreSQL)

Tabela: perfis (Armazena as preferências do usuário para personalização)

- **id** (UUID, Primary Key): Identificador único do usuário (vinculado ao Supabase Auth).
- **nome** (Text): Nome de exibição.
- **nivel_experiencia** (Text): "Iniciante", "Intermediário" ou "Avançado".
- **condicao_fisica** (Text): Descrição breve da capacidade física.
- **interesses_usuario** (JSON/Array): Lista de tags (ex: ["Trilhas", "Praias", "Cachoeiras"]).

10.2. Banco Vetorial (ChromaDB)

Coleção: ecorroteiro_locais

- **ID**: Identificador único do documento.
- **Embedding**: Vetor denso (384 dimensões) gerado pelo modelo all-MiniLM-L6-v2.
- **Metadata**:
 - **title**: Nome do local.
 - **difficuldade**: Nível de esforço.
 - **json_data**: O objeto JSON completo do roteiro para recuperação rápida.
- **Document**: O texto completo (full_search_text) usado para a busca semântica.

11. Especificação da API (Endpoints)

O Backend expõe uma API RESTful documentada. Abaixo, os principais recursos:

Método	Endpoint	Descrição	Proteção

POST	/signup	Cria um novo usuário no Supabase Auth e na tabela de perfis.	Pública
POST	/login	Autentica o usuário e retorna o Token JWT de sessão.	Pública
GET	/profile	Retorna os dados do perfil do usuário logado.	Bearer Token
PUT	/profile	Atualiza interesses e nível de experiência do usuário.	Bearer Token
POST	/chat	Envia mensagem para a IA. Executa o pipeline RAG e retorna a resposta.	Bearer Token
POST	/routes/suggest	Gera 3 sugestões de roteiros estruturados em JSON baseados no perfil.	Bearer Token